

Progress Report

Evaluation Data Generation on the Fly

Denis Maxheimer, Beyza Ünlü, Niklas Zoz

March - April 2026

Contents

1	Introduction	2
2	Grammatical Error Correction	2
2.1	Generators	2
2.2	Metrics	3
2.2.1	Errant	3
2.2.2	GLEU	3
2.2.3	CoLA Score	3
2.2.4	Correction extent	4
2.3	Evaluators	4
2.3.1	T5 Grammar Correction (Vennify)	4
2.3.2	Grammar Error Correcter V1 (Gramformer)	4
2.3.3	CoEdit-Large (Grammarly)	4
2.3.4	Claude Haiku	4
3	Framework Architecture	5
3.1	The GET Execution Lifecycle	5
4	Evaluation	6
4.1	Prompt Engineering	7
5	Challenges	8
5.1	Data Complexity	8
5.2	Computational Intensity	9
6	Outlook	9

1 Introduction

The rapid advancement of Large Language Models (LLMs) has outpaced traditional evaluation methods, which often rely on static, memorized benchmarks. To combat this "Vicious Circle of AI Evaluation," our team aims to implement the suggested GET Framework. Unlike traditional benchmarks tied to specific datasets, GET is designed as a universally applicable architecture capable of dynamically assessing model performance across any Natural Language Processing (NLP) task. The framework is designed to be modular, separating the core execution logic from task-specific requirements. It operates through a three-stage lifecycle:

- **Generate:** Creating high-quality, task-specific synthetic data on the fly using advanced LLMs to ensure the evaluation material is always unique.
- **Evaluate:** Subjecting target models to these unique challenges and scoring them against real-time metrics.
- **Trash:** Immediately disposing of synthetic data to ensure benchmark volatility, preventing the data from ever leaking into future training sets.

In our first sprint (March-April 2026), we have validated this universal architecture using Grammatical Error Correction (GEC) as our primary proof-of-concept. While GEC serves as our initial testing ground, the framework is built to be extended to diverse tasks such as Hate Speech Detection or Summarization by simply swapping the task-specific modules. This report details our progress setting up a scalable infrastructure, as well as current findings, challenges, and finally an outlook for the following project timeline.

2 Grammatical Error Correction

To build our framework, we first decided to create an approach for one dedicated topic. We chose GEC for this purpose, as it was specifically suggested in the project outline as a suitable task for a generic evaluation framework. This task allowed us to get an initial understanding for the task and implement the GET Method in a small scope. We implemented required generators, metrics and evaluators, which are required to achieve a holistic functionality for the GET Lifecycle.

2.1 Generators

The framework is designed to be model-agnostic. For example, currently we are utilizing minimax-m2.7 accessed via MiniMax¹ to generate our synthetic data. The generator produces synthetic sentences for our GEC benchmark. Rather than generating sentences from scratch, our implementation uses a seed-based

¹<https://www.minimax.io>

approach. We extract samples from an existing dataset, which are then passed to the generator LLM to introduce specific grammatical errors while maintaining the original sentence’s context.

A more in-depth description regarding our prompt instructions and specific prompt engineering approaches will be described in Section 4.1.

2.2 Metrics

To provide a robust and multi-dimensional assessment of the task models within our framework, we selected ERRANT and GLEU as our primary benchmarking metrics for GEC.

2.2.1 Errant

The ERRANT Scorer (ERRor ANnotation Toolkit) ² is a reference-based metric that measures the performance with an edit-based F-Score. Furthermore, ERRANT is able to calculate error type scores through an rule-based error type framework.

It evaluates this error types regarding three different levels of granularity:

- **Edit Operation (3 labels):** Missing, Replacement, Unnecessary
- **Main Type (25 labels):** e.g., Noun, Spelling, Verb Tense
- **Full Type (55 labels):** e.g., Missing Noun, Replacement Noun, Unnecessary Noun

2.2.2 GLEU

GLEU (General Language Evaluation Understanding) ³ is also a reference-based metric. Unlike other benchmarks, it requires only corrected reference sentences and no explicit edit annotations. GLEU rewards hypothesis n -grams that overlap with the reference but were absent from the original text. Furthermore, it penalizes hypothesis n -grams that overlap with the original text but not the reference. To ensure the accuracy of the results, a brevity penalty is included to prevent the model from generating artificially short sentences to exploit the precision calculation and inflate its score.

2.2.3 CoLA Score

The Corpus of Linguistic Acceptability (CoLA) ⁴ score provides a deep-learning-based assessment of a sentence’s grammatical integrity. The model processes a sentence and returns a binary label whether the sentence is grammatically correct or not. We monitor this at three points: cola_A (original seed), cola_B

²<https://github.com/chrisjbryant/errant>

³<https://github.com/cnap/gec-ranking>

⁴<https://github.com/nyu-ml/CoLA-baselines>

(synthetic corruption), and cola_C (final correction). This captures the full $A \rightarrow B \rightarrow C$ lifecycle by quantifying the grammatical impact of each stage.

2.2.4 Correction extent

Correction Extent quantifies the volume of structural change by calculating the $1 - \text{bigram Jaccard distance}$ between the model’s input and its prediction. A score of 0.0 indicates the model returned the input verbatim, while higher values represent more aggressive rewriting behaviors. This metric is paired with cola_delta to distinguish between edits that made a sentence fluent and edits that were unnecessary.

2.3 Evaluators

In the context of the framework, the evaluators serve as the target models whose performance is measured against our generated synthetic benchmarks and existing benchmarks. To ensure a comprehensive assessment, we selected four distinct models that specialize in different aspects of Grammatical Error Correction (GEC).

2.3.1 T5 Grammar Correction (Vennify)

T5 Grammar Correction⁵ is a fine-tuned version of Google’s T5-base model. This model primarily focuses on fluency and sentence rewriting.

2.3.2 Grammar Error Correcter V1 (Gramformer)

Grammar Error Correcter V1 (Gramformer)⁶ is a specialized GEC tool built on a T5 architecture but trained on a more diverse corpus, including filtered WikiEdits and synthetic data.

2.3.3 CoEdit-Large (Grammarly)

CoEdit-Large⁷ is a text-to-text transformer fine-tuned by Grammarly specifically for text editing tasks. Unlike traditional GEC models, CoEdit is designed to handle multiple editing intents, such as fluency, paraphrasing, and formalization. In the context of our framework, we utilize this model as a high-capacity evaluator to test its performance against complex synthetic errors.

2.3.4 Claude Haiku

We use Claude Haiku to represent the class of general-purpose instruction-tuned LLMs used for zero-shot evaluation. We utilize Haiku to analyze how a model with broad linguistic reasoning compares to task-specific architectures like T5 or

⁵<https://huggingface.co/vennify/t5-base-grammar-correction>

⁶https://huggingface.co/prithivida/grammar_error_correcter_v1

⁷<https://huggingface.co/grammarly/coedit-large>

CoEdit-Large, specifically in terms of its ability to identify and rectify synthetic errors without task-specific fine-tuning.

3 Framework Architecture

To ensure the scalability and maintainability of our GET framework, we developed a modular project structure that separates concerns between data generation, model evaluation, and metric calculation. This architecture allows us to easily extend the framework to new tasks (e.g., Hate Speech or Spam Detection) by simply implementing new task-specific logic while keeping the core pipeline intact. Our directory is structured as follows:

```
framework/
├── main.py ..... entry point
├── pipeline.py ..... GET loop (Generate, Evaluate, Trash)
├── requirements.txt ..... Python dependencies
├── configs/
│   ├── config.yaml ..... dataset, generator, task models, output
│   └── tasks/
│       └── gec.json ..... GEC task config (error types, prompts, metrics)
├── tasks/
│   ├── base_task.py ..... abstract task template
│   └── gec/
│       └── task.py ..... Grammatical Error Correction task logic
├── generators/
│   ├── openai_generator.py ..... OpenAI, Groq, OpenRouter, Mistral
│   ├── anthropic_generator.py
│   └── google_generator.py
├── evaluators/
│   └── gec/
│       ├── t5.py
│       ├── gec_v1.py
│       ├── coedit.py
│       └── claude.py
├── metrics/
│   ├── gleu.py ..... GLEU score
│   └── gec/
│       ├── errant.py ..... ERRANT (P, R,  $F_{0.5}$ )
│       ├── cola.py
│       ├── correction
│       └── n_edits.py
```

3.1 The GET Execution Lifecycle

The main logic of our framework is handled in the `pipeline.py`, specifically the `run_pipeline` function. It is designed to be fully reproducible and modular, allowing for different task and generator configurations via the `config.yaml`

file. The execution can be described with the following steps:

1. **Synthetic Generation:** The pipeline begins by loading real world samples which serve as a thematic foundation. These initial samples are then transformed by an LLM-based generator into M unique, task-specific synthetic variations.
2. **Quality Verification:** Each generated sample is passed through a `verify()` method to ensure the LLM applied the requested corruptions while maintaining linguistic validity. Furthermore, an LLM-as-a-judge performs a qualitative audit of the synthetic output, filtering out trivial or illogical errors to ensure the final benchmark maintains high structural complexity and semantic integrity.
3. **Evaluation:** The verified data is fed into the target models (e.g., T5 or Gramformer), where performance is measured using task-specific metrics like GLEU and ERRANT.
4. **Benchmark Trashing:** To maintain benchmark volatility, the framework executes `synthetic.clear()` after every run, ensuring the generated data only exists for the current run, not leaking into future training sets.
5. **Statistical Aggregation:** Upon completing N iterations, the `aggregate()` function calculates the mean and standard deviation (σ) to provide a robust measure of model stability.

4 Evaluation

The results shown in Figure 1 were produced through three iterations, with a sample size of 20 corrupted sentences per iteration. The evaluation of these results shows clear differences between the models and their standard deviation.

CoEdit-Large emerged as the most reliable benchmark, achieving the highest $F_{0.5}$ score (0.8400) with a remarkably low standard deviation (0.0105), indicating that its linguistic precision is consistent regardless of the specific synthetic batch. In contrast, the T5-Base model exhibited significantly higher volatility ($std = 0.0627$). This suggests that the framework successfully triggers specific model vulnerabilities and exposes edge cases that smaller architectures struggle to handle consistently.

Crucially, the framework’s ability to take inputs with low linguistic acceptability (CoLA = 0.488) and facilitate their correction to near-perfect levels (CoLA = 0.960) demonstrates that the synthetic data produced is both challenging and linguistically recoverable. While the observed variance in smaller models suggests that further scaling of the sample size could refine the scores, these results fundamentally highlight the framework’s capability to produce high-quality, consistent synthetic data for rigorous benchmarking.

```

vennify/t5-base-grammar-correction:
gleu: 0.7584 ± 0.0325
errant.precision: 0.7786 ± 0.0536
errant.recall: 0.5697 ± 0.0849
errant.f0.5: 0.7239 ± 0.0627
errant_dist: 0.7519 ± 0.0392
cola.input: 0.488 ± 0.0956
cola.corrected: 0.9248 ± 0.0254
cola.delta: 0.4368 ± 0.1094
correction_extent: 0.4501 ± 0.0342
n_edits: 2.439 ± 0.2466

```

(a) vennify/t5-base-grammar-correction

```

grammarly/coedit-large:
gleu: 0.8357 ± 0.015
errant.precision: 0.8783 ± 0.018
errant.recall: 0.7172 ± 0.0318
errant.f0.5: 0.84 ± 0.0105
errant_dist: 0.8433 ± 0.036
cola.input: 0.488 ± 0.0956
cola.corrected: 0.9608 ± 0.0554
cola.delta: 0.4727 ± 0.0401
correction_extent: 0.4781 ± 0.0413
n_edits: 2.7745 ± 0.4775

```

(b) grammarly/coedit-large

Figure 1: Example results for two evaluator models.

4.1 Prompt Engineering

To ensure the use of the highest-quality prompts for data generation, five distinct prompt strategies were tested to determine their impact on the ERRANT $F_{0.5}$ score:

- **Zero-Shot Baseline:** Instructions only, forcing the model to infer the task without examples.
- **One-Shot:** Provides a single example to clarify the expected input/output format.
- **Few-Shot Stratified (3-shot vs. 12-shot):** Tests whether a balanced set of frequent errors (3-shot) or full taxonomy coverage (12-shot) improves results.
- **Chain-of-Thought (CoT):** Requires the LLM to explicitly identify the error type before generating the sentence, acting as a linguistic anchor.

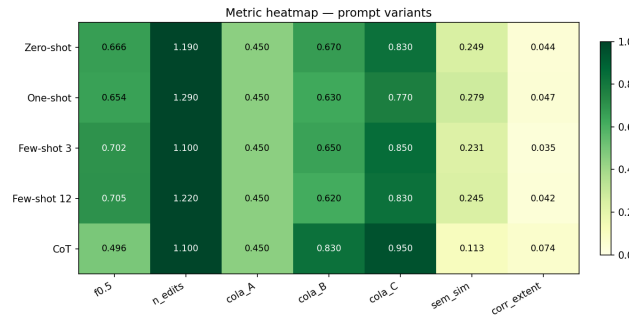


Figure 2: Metric heatmap for the prompt variants.

As shown in Figure 2, the 12-shot variant achieved the highest precision baseline ($F_{0.5}$: 0.705), providing a robust empirical reference for consistent error generation. CoT, however, is utilized as our primary generation strategy due to its superior linguistic reasoning and output stability. Despite a lower $F_{0.5}$ (0.496), CoT produces the highest final grammatical acceptability (`cola_C`: 0.950) and structural complexity (`corr_extent`: 0.074).

To further optimize the generators, we conducted an ablation study to identify the most effective prompt and model configurations. As highlighted in Figure 3, we concluded that a low temperature of 0.3 is essential for maintaining focus, resulting in a superior accuracy score of 0.657. We also discovered that concise prompts (0.640) significantly outperformed detailed instructions (0.572), which appeared to introduce unnecessary noise.

These findings further highlight the trade-off between accuracy and structural complexity. While the 12-shot variant provides the most reliable baseline for precision, the CoT strategy is superior for generating linguistic complexity and recoverable errors.

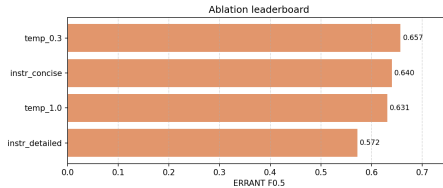


Figure 3: Ablation results.

5 Challenges

This section outlines the primary obstacles encountered during the development and execution of the GET framework, focusing on the discrepancy between synthetic and natural error distributions and the operational bottlenecks of high-latency generation.

5.1 Data Complexity

One of the most significant challenges identified by the GET framework is the measurable difference in data complexity between natural and synthetic error distributions. As illustrated in Figure 4, evaluation models consistently achieve higher scores on generated data compared to the original reference dataset. For example, the ERRANT $F_{0.5}$ score for CoEdit-L increases from 0.536 on natural data to 0.679 on the generated benchmark. This performance gap suggests that while the framework successfully creates linguistically recoverable errors, the synthetic outputs may currently lack the idiosyncratic complexity, noise, and variety of error types found in natural datasets such as the FCE. Moving

forward, our main priority will be optimizing these generated errors so they more closely mimic real human behavior. We also need to investigate whether this complexity gap is unique to GEC or if it’s a broader issue with synthetic data in general.

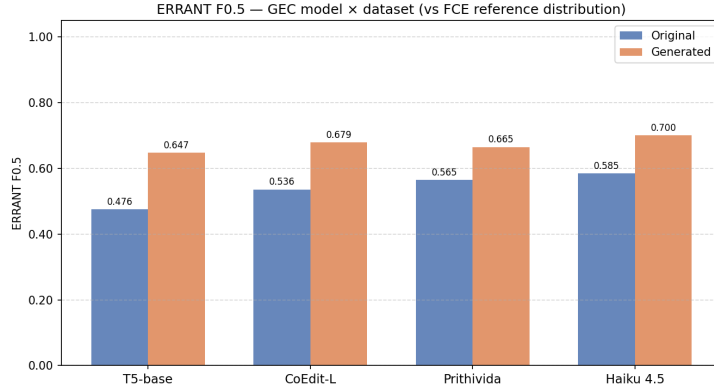


Figure 4: Comparison between synthetic and original data.

5.2 Computational Intensity

Our framework is quite resource-intensive when it comes to time and financial costs. Because of the introduction of a Judge, to increase the data quality every single sentence requires at least two API calls, one to generate the error and the other one to verify the corrupted sentence. This effectively doubles our token usage and costs. On the local side, running heavy evaluation models like T5 for correction and BERT for CoLA scoring adds extra computational strain and hardware requirements to the pipeline. These factors combined make large-scale benchmarking slow and expensive, forcing us to be very selective with our sample sizes and test runs. Therefore, we will try to come up with solutions to either run the generator models locally or investigate alternative options to reduce computational complexity.

6 Outlook

The successful validation of the GET framework within the GEC domain provides a robust foundation for the upcoming project phases. Our primary objective moving forward is to transition the framework from a GEC pilot into a universally applicable evaluation suite.

As outlined in Table 1, Sprint 2 will prioritize addressing the challenges identified during our initial runs. This includes optimizing the execution logic to reduce computational intensity and refining our generation strategies to bridge the complexity gap between synthetic and natural data. Furthermore, we will

diversify the functionality of the framework by introducing Hate Speech and Spam detection. 1.

Sprint	Status	Activities & Goals
Sprint 1 (03-04.2026)	Completed	• Developed GET pipeline. • Implemented a functional GEC benchmark. • Optimized prompts for synthetic data generation.
Sprint 2 (05-06.2026)	Next	• Hate speech and Spam detection. • Increase data complexity. • Reduce computational intensity.
Sprint 3 (07-08.2026)	Planned	• Sentiment analysis. • Expand to further NLP tasks.

Table 1: Roadmap